

Generative Models and Sequence Modelling

Introduction to Deep Learning Assignment 2

Yutao Liu (s4213211)

Nael Belhaj Haddou (s4431278)

1 Task 1: Generative Models

1.1 Instructions

This experiment requires us to learn and implement image generation tasks by using generative models such as Variational Autoencoders (VAE) and Generative Adversarial Networks (GAN). This includes exploring the latent space distribution of the model, generating new images, and analyzing the impact of hyperparameter adjustment and network architecture optimization on the generation quality.

We will learn about the basic architecture and construction principles of VAE and GAN by following the code in the provided Jupyter Notebook. After that, we will find and preprocess a new image dataset on the network and adjust it to an input size suitable for the experiment (64x64x3). The model is trained using the preprocessed dataset and the model performance is visualized by generating images and latent space interpolation. Finally, the network architecture and hyperparameters are modified to improve the generation quality of the model and the changes brought about by the adjustments are analyzed.

1.2 Experiment

1. Selection of dataset

- (a) Faces in the Wild (LFW) is a database of face photos. 1,680 people in the photos have two or more different photos in the dataset. The original database contains four different sets of LFW images and three different types of "aligned" images. We choose this database as the training set for this experiment.

Visit *the Kaggle LFW Dataset* for more information about the database.

2. Dataset preprocessing

- (a) Use Python to write a script to preprocess the image data, resize the images in the database to the specified size (64x64), and save them in .npz format.

3. VAE and GAN Training

- (a) Train the model using the preprocessed dataset and visualize the model performance by generating images and latent space interpolation.

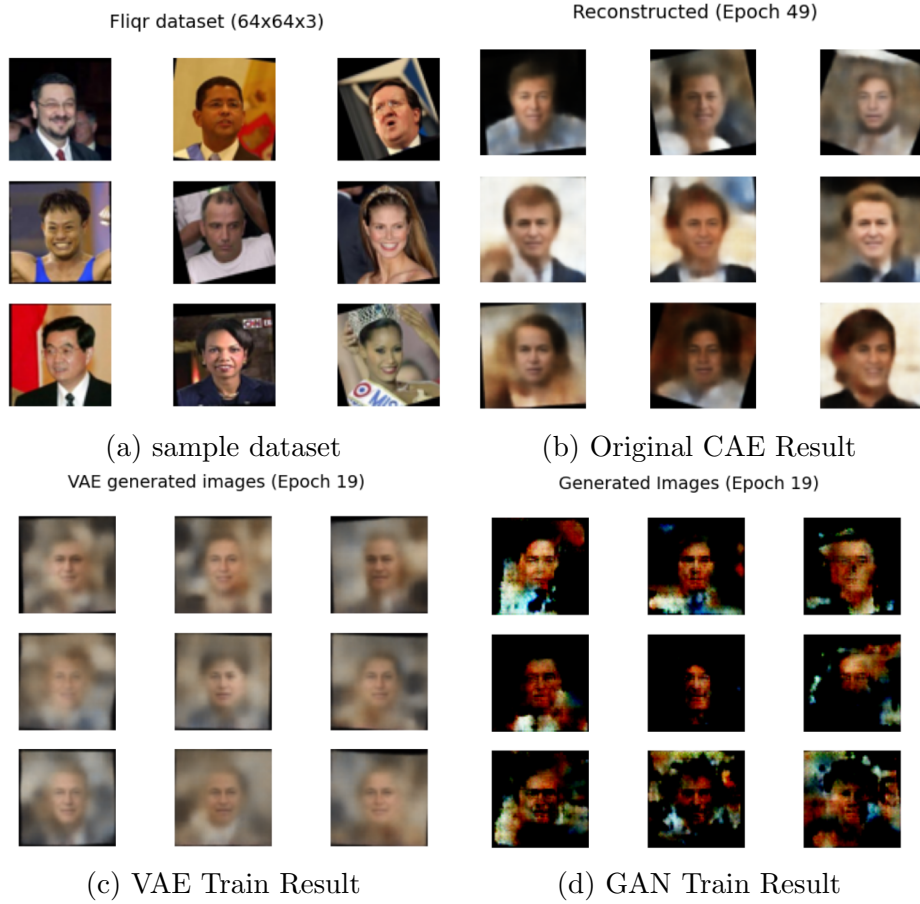


Figure 1: The dataset is loaded and trained correctly

1.3 Observations

After the dataset is properly loaded and trained, we can modify the network architecture and hyperparameters to improve the generative quality of the model.

1.3.1 Deepening of network architecture

1. Encoder:

- (a) Increase the original four convolutional layers to six layers; A fixed number of filters is set in the initial layer. As the network depth increases, the number of filters increases layer by layer. The number of filters in the first three layers is 128, 256, and 512, respectively, and remains unchanged from the fourth layer.
- (b) Added Batch Normalization to improve training stability.

2. Decoder:

- (a) The number of filters is halved from 512 to 64;
- (b) A reshape layer is added to convert the latent vector into the corresponding feature map

format.

1.3.2 Variational Autoencoder (VAE)

1. Latent Space Visualization Function: Randomly generate two points from the latent space, and perform linear interpolation between the two points to generate multiple sets of latent variables. Finally, use the decoder to decode these latent variables into images to demonstrate the linear interpolation effect of the latent space;
2. The loss function uses mean square error (MSE) instead of the original Binary Crossentropy to reconstruct the global image structure;
3. Customize the KLLossLayer, dynamically adjust the weight `kl.weight` of the KL loss, and gradually increase the importance of KL with the number of training rounds;
4. Adjust the `laten_dim`, `filters`, and `epochs` parameters to achieve the best results.

Results:

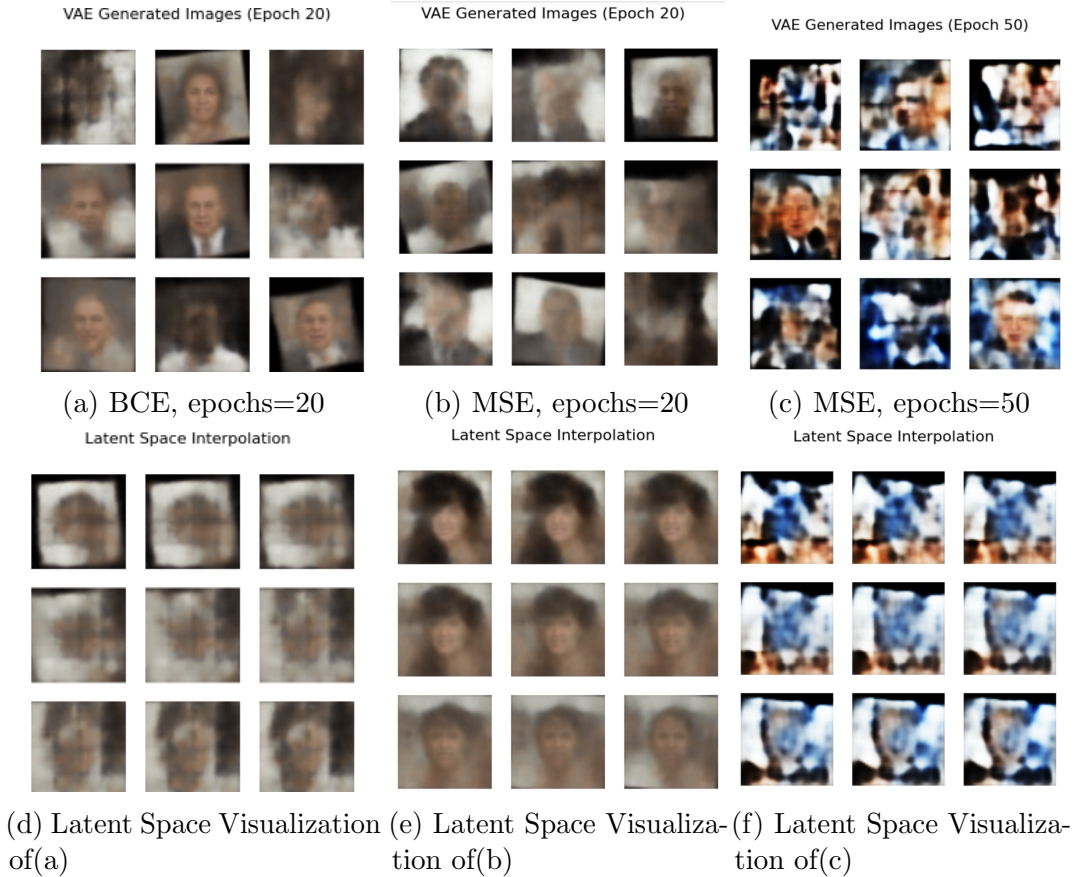


Figure 2: VAE training and parameters tuning

It can be seen that the image quality is low at the beginning, and only very blurred facial contours can be seen. This is because when using binary cross entropy (BSE) as the loss function, due to its

high sensitivity to detail pixels, it is easy to cause the generated results to be unstable or overly blurred in the initial training; after changing the loss function and increasing the value of latent_dim, the image quality is slightly improved, the contours are clearer, and the color distribution is more natural; after further increasing the number of training rounds, the facial structure is clearer, and details (such as the position of facial features) are better preserved. At the same time, the MSE loss function can converge stably, and the generated image is closer to the real distribution.

1.3.3 Generative Adversarial Networks (GANs)

1. Generator is based on the improved decoder implementation, with deeper layers and larger potential dimensions(default 256);
2. The optimizer uses a reduced learning_rate for better GAN training;
3. The discriminator freezes parameters when training GAN while dynamically adjust the generator and discriminator weights to prevent training instability;
4. Adjust the laten_dim, filters, and epochs parameters to achieve the best results

Results:

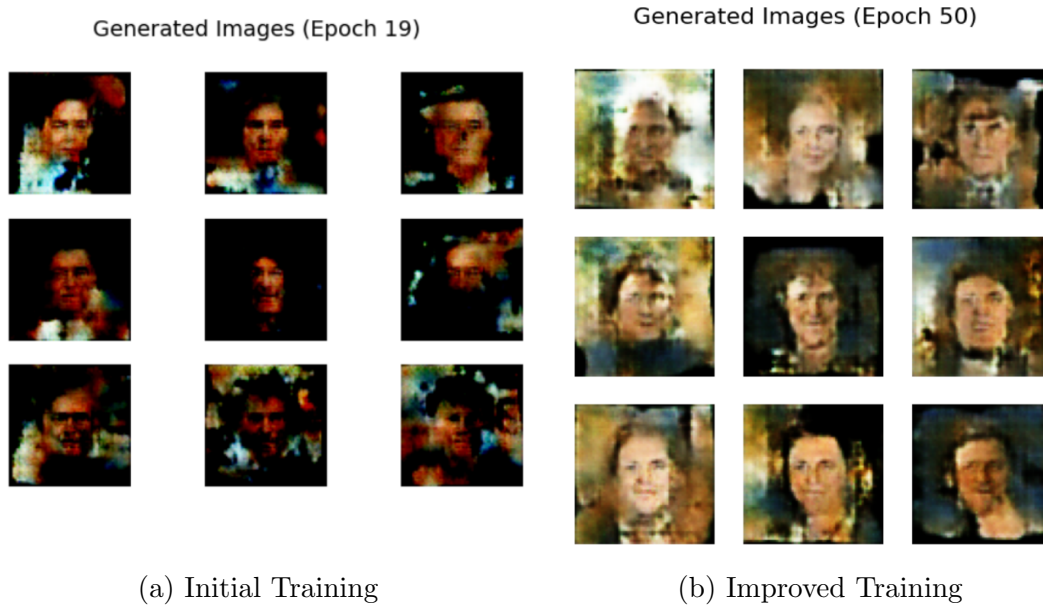


Figure 3: GAN training and parameters tuning

Through comparison, it can be found that the image quality obtained by the improved training has been significantly improved, and the shape of the face and clearer features (such as eyes, mouth) can be clearly seen. The color distribution of the image is more reasonable, the noise is reduced, and the distinction between the background and the foreground becomes clearer; some generated faces still have slight distortion or blur, but the overall level is close to identifiable. This shows that the generator has learned more data distribution characteristics and can generate more meaningful samples.

1.4 Conclusion

1.4.1 Overall Comparison

1. GAN outperformed VAE in generating sharper images, owing to its adversarial loss, which explicitly pushes the generator to produce realistic outputs.
2. While both models showed reasonable diversity, the VAE demonstrated smoother latent space transitions, while GAN outputs were occasionally repetitive.
3. VAE training was more stable compared to GAN, which required careful tuning of hyperparameters to prevent instability.

1.4.2 VAE Evaluation

1. The optimized VAE generated images that showed clearer structural coherence compared to earlier iterations. The incorporation of a hierarchical architecture with Batch Normalization and Dropout improved the model’s stability and generalization.
2. Latent space interpolation results displayed smooth transitions, indicating that the latent space was well-structured, enabling meaningful interpolations between generated samples.
3. In the future, we still need to adjust the balance between KL divergence and reconstruction loss. At the same time, using perceptual loss functions to enhance image clarity.

1.4.3 GAN Evaluation

1. The optimized GAN model demonstrated stability during training, thanks to architectural refinements and careful tuning of the learning rate and batch size.
2. Occasional artifacts and repetitive outputs indicate further improvements are needed to capture the full data diversity.
3. In the future, we could adopt advanced GAN variants such as WGAN-GP or StyleGAN which can significantly improve both stability and quality.

In conclusion, both VAE and GAN show strengths and limitations in this task. VAE excels in latent space organization and interpolation, making it suitable for applications that require structured and interpretable latent spaces. On the other hand, GAN produces more visually appealing results, making it a better choice for tasks that prioritize output realism. For future projects, we can combine the strengths of both methods to produce high-quality and interpretable results, pushing the boundaries of generative models in real-world applications.

2 Task 2: Sequence modelling with recurrent neural network

2.1 Instructions

This assignment required building recurrent neural networks for modeling sequences in image and text datasets. Additional symbols, a plus and minus, were generated and added to the data. The dataset was converted into two formats: sequences of characters (and corresponding one hot matrices) and corresponding images. The data represents additions or subtractions of pairs of at most 2 digit numbers and the objective values represent the result of the corresponding calculation. This assignment tasked us with building recurrent neural networks, apt to modeling sequences, to use on both the image and text datasets. We worked on three types of models, text-to-text models, image-to-text models and finally text-to-image models. We built 2–4 models for each task, analyzed their mistakes, and compared their performance.

2.2 Experiment

We used a fixed random seed in our experiments to improve reproducibility. All models were trained using the ADAM optimizer and its default hyperparameters in the keras implementation. When unspecified, the activation function for a layer is ReLU and the padding for all convolution and transpose convolutions is same. For each model, we report several metrics, including final test set accuracy (when applicable), mistake ratios by operation and output sign, mistakes by digit presence in the input or output, and mean absolute error. Some predictions, such as '-' or '-7,' cannot be converted to numbers. For mean absolute error calculations they are treated as 0. As a result, our mean absolute error measurements may be overestimated.

2.2.1 Text to Text Models

The first three text to text models we tested were built using the provided code. They all consist of one 256 cells LSTM layer for the encoder. Their decoders are made of one 256 cells LSTM layer and a time distributed 13 neurons dense layer (with a softmax activation function) as output. We test this architecture using three different train and test splits: 50/50, 25/75 and 10/90. We also test a different architecture with 3 LSTM layers in the encoder and the same decoder on a 25/75 data split. All these models are trained for 20 epochs, using a categorical crossentropy loss function.

2.2.2 Image to Text Models

We built three image to text models. The first model's encoder consisted of 3 convolutional LSTM layers with 3x3 kernels and 32, 64 and 128 filters respectively. Each layer was followed by 1x2x2 max pooling and batch normalization. The decoder used five 512 cells LSTM layers with 0.5 dropouts in between them. The other two models used the same decoder but were designed for sequences of flattened images. Their encoders began by reshaping the data into flattened images, followed by three and five 256 cell LSTM layers respectively, each with 0.5 dropout. These models were trained with a categorical crossentropy loss on an 64/16/20 split. Training lasted up to 100 epochs,

with early stopping used to halt training if validation loss did not improve for 10 consecutive epochs.

2.2.3 Text to Image Models

We trained two text to image models. The first one uses three 256 cells LSTM layers for the encoder, while the second one uses 6. Both models share the same decoder structure: a 784 cells LSTM layer, followed by a reshape into a 3x28x28 array, followed by 4 transposed convolution layers using 128, 64, 32 and 3 filters respectively. These models are trained for 50 epochs using mean squared error loss using a 64/16/20 data split.

We first showcase some of the output of each model. Since these models produce image outputs, we used a separate model to map images back to their corresponding text. This model consisted of three 256-cell LSTM layers and a time-distributed dense layer with 13 neurons as the output. We trained it using categorical crossentropy for 20 epochs on the full data. But, to ensure the validity of our metrics we first measure the score of this special image to text model on an 80/20 train and test split.

2.3 Observations

To save space some numbers are reported without an associated figure. The corresponding figures and a table containing model metrics can be found in the appendix.

Models make more mistakes when 1 appears in the output, whether due to a higher frequency in the data or inherent modeling difficulty is unclear.

2.3.1 Text to Text Models

As expected, the models trained on more data were more accurate. With the best performing model, the 50/50 split mode, achieving an 80% accuracy on test data. It is also worth noting that deepening the encoder also led to a higher accuracy, with the 25/75 model reaching 62% and the deeper encoder model reaching 68% test accuracy. However, accuracy alone is not informative enough. To make conclusions we must compare mean absolute errors too. The mean absolute errors we observe are 2.07, 6.83, and 8.04 for the 50/50, 25/75, and 10/90 data splits respectively. Meanwhile, the deeper encoder model reached a test mean absolute error of 3.77.

While the difference in accuracy between the 25/75 model and the 10/90 and deeper 25/75 models is similar (about 6 points). The difference in mean absolute error is much larger, 1.2 between 25/75 and 10/90 and 3.06 compared to the deeper encoder model. Similarly, The 50/50 model and deeper model have a 33% smaller accuracy gap than the 25/75 and 50/50 models, but their mean absolute error difference is over 50% smaller. Meanwhile, the difference in their mean absolute errors is more than halved. This means that deepening the encoder may have led the model to capture more subtle rules linking input and output data as the smaller increase in accuracy suggests this change does not only stem from the model memorizing the data.

As can be seen in figures 4, 5, 6, and 7, the models did not make much more mistakes depending on the digits present in the input. However, it appears that more accurate models tend to make more mistakes on subtractions. This could mean that as the model captures the data worse and worse

(as shown by the accuracies and mean absolute errors), it gets worse at the more complex operation faster. It is also notable that the deeper encoder model makes less mistakes on additions than the 25/75 mode. As all these metrics are for test datasets, we can likely eliminate the possibility that these changes are caused by better memorization as this is new data to the model. These elements and the differences in accuracy and mean absolute error lead us to conclude that the deeper encoder can help a model that underfits on this task. Allowing it to understand rules of the underlying operations better.

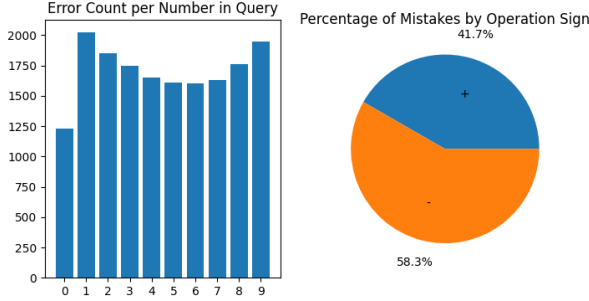


Figure 4: Mistake counts and ratios depending on input for the 50/50 data split model

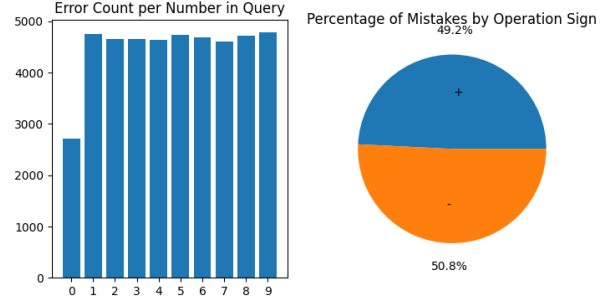


Figure 5: Mistake counts and ratios depending on input for the 25/75 data split model

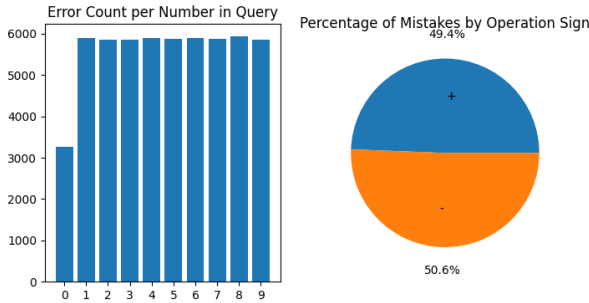


Figure 6: Mistake counts and ratios depending on input for the 10/90 data split model

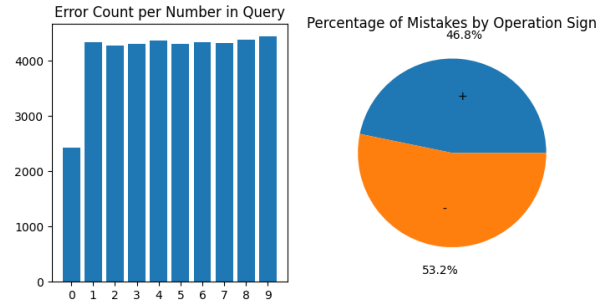


Figure 7: Mistakes counts and ratios depending on input for the deeper encoder model

2.3.2 Image to Text Models

The convolutional model performed a little better than the one flattening images. It achieved an accuracy of 49.9% compared to 47.6%. Both models appear to be making similar mistakes, although the convolutional model's mistake ratio was slightly more dominated by subtractions. This leads us to consider the flattened as almost better because while it trained for 40 epochs compared to the convolutional models' 32. Each epoch of the flattened image model took less than half as long than a convolutional model's epoch (24s on average against 74s). This means the flattened model took less than half as long to train for almost equal results.

Comparing the flattened image model to its deeper encoder counterpart yields more interesting results. The deeper model only achieved an accuracy of 32.55% and a mean absolute error of 60.86.

This is almost 6 times the mean absolute error of the base flattened image model. Meanwhile, their mistakes per number were distributed similarly. Although the mistake ratios depending on operation still followed the trend we observed in the text-to-text model, with better performing models making relatively less mistakes in additions. This and the text-to-text results lead us to believe that while different operations have different complexities, a digits presence in the input does not impact the operation’s complexity much.

2.3.3 Text to Image Models

On an 80/20 data split, the model achieved a 98% test accuracy and a 1.73 test mean absolute error. We consider these results good enough to use this model on the text-to-image outputs. While these metrics are quite good but should be kept in mind as inaccuracies in numbers reported for the rest of this section.

In figures 8, 9, and 10 and figures 11, 12, and 13 we can compare some output examples. It was expected that the outputs would be blurry as this is not a proper generative model architecture and both models were trained on mean squared error. The model was trained on pixel values, meaning that neither its training data nor loss reflect mean absolute error. Consequently, we expected it to have a far worse mean absolute error than other models but a possibly competitive accuracy. However the results reflected the opposite, even when accounting for the uncertainty stemming from the model that analyzes output images. The accuracy was 8.09%, less than a third as accurate as the third worst model. On the other hand, the mean absolute error was 15.02, four times smaller than the worst. Another observation that could help explain this is that the model seems to be

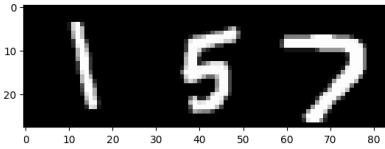


Figure 8: Correct output example 1

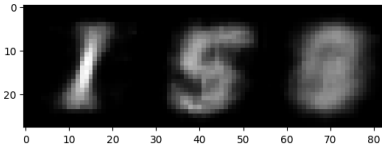


Figure 9: Base text-to-image model output example 1

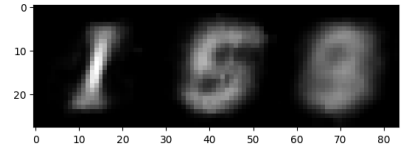


Figure 10: Deeper encoder model output example 1

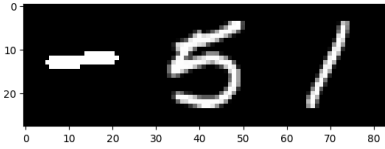


Figure 11: Correct output example 2

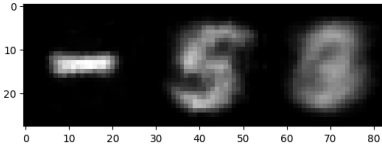


Figure 12: Base text-to-image model output example 2

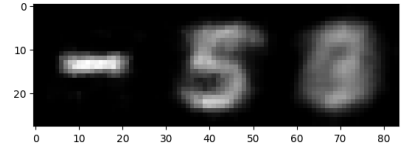


Figure 13: Deeper encoder model output example 2

more precise on symbols further left in the output. This is reflected in the leftmost symbols often being less blurry. We are currently unsure what causes this difference. Our main theory is that the model captures output sequences worse, thus performing worse for output deeper in the sequence. Although this does not explain what in the model architecture is the cause.

Deepening the encoder only increasing mean absolute error by about 25% and lowering accuracy by 0.84 points, relatively small changes. It also makes almost no visible difference in the outputs we observed, just making some numbers slightly less blurry. This is surprising as the deeper model

has worse performance metrics, leading us to believe it has a better looking output. This could mean both models may make similar mistakes, but only the clearer outputs are recognized.

2.3.4 Encoder Depth

In the previously mentioned experiments deepening a model’s encoder has yielded very variable results. This indicates that depending on the task there is an optimal number of encoder layers. This conclusion is supported by these results where models with deeper encoders had 45% less to 429% more mean absolute error. Incredibly large swings in performances in both directions with the addition of just 2 or 3 encoder layers. The influence of other factors remains to be tested such as decoder depth or input size.

2.4 Conclusion

1. Main takeaways:

- (a) The text-to-text experiments showed how changes in train and test data splits influence model performance. A better understanding of the relationship between training set size and a model’s ”understanding” of the relationship between inputs and outputs is crucial in designing more effective experiments and better performing models.
- (b) In the context of two digit additions and subtractions. Subtractions are noticeably more complex for these models. On the other hand, the digits contained in the input have little to no influence on the task’s difficulty.
- (c) The results on encoder depth differ from typical overfitting, where training performance improves but test performance declines. This pattern was absent, even with deeper encoders. This can be a trap when building architectures and shows the complexity of sequence-to-sequence recurrent neural networks. This may mean the encoded data’s distribution was too complex for the decoder model to learn.
- (d) When it improves performances, deepening the encoder architecture seems to not only improve memorization abilities. But also allow tested models to capture more complex underlying rules better.

2. Directions for future work

- (a) Because sequence-to-sequence recurrent neural networks are more complex as they have 2 modules (encoder and decoder). It can be difficult to create effective architectures for inexperienced users. For this reason, understanding what parameters are most important and how they tend to influence the model can be invaluable. Thus, it would be interesting to build an inventory of parameters and compare how influential they are in this type of model’s performances.
- (b) While these experiments highlighted the possible consequences of different encoder depths. We could not explore all the factors that influence the optimal depth in depth. Finding the parameters that change this optimum and understanding their influence could help practitioners save a lot of time during architecture search.

3 Appendix

3.1 Task 2

Model	Test Accuracy	Test Mean Absolute Error
Text-to-Text 50/50 split	80.47%	2.07
Text-to-Text 25/75 split	62.08%	6.83
Text-to-Text 10/90 split	55.31%	8.04
Deeper Encoder Text-to-Text	68.24%	3.77
Convolutional Image-to-Text	49.91%	9.82
Flattened Image-to-Text	47.60%	11.5
Deeper Encoder Image-to-Text	32.55%	60.86
Image Output to Text	98.32%	1.73
Text-to-Image	8.09%	15.02
Deeper Encoder Text-to-Image	7.25%	18.84

Figure 14: Table of our output metrics for each model

3.1.1 Text to Text

3.1.2 Image to Text

3.1.3 Text to Image

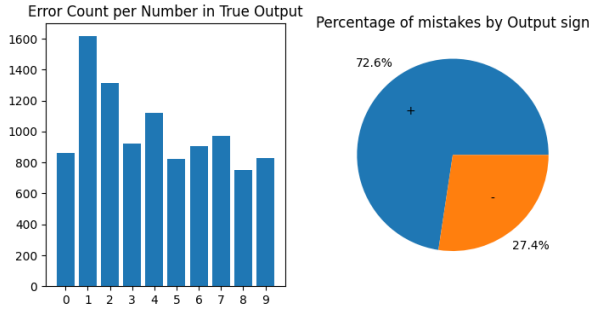


Figure 15: Mistake counts and ratios depending on output for the 50/50 data split model

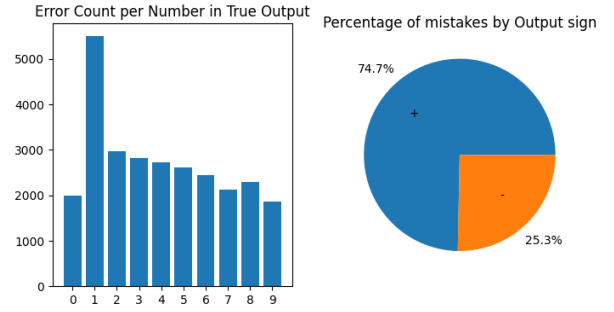


Figure 16: Mistake counts and ratios depending on output for the 25/75 data split model

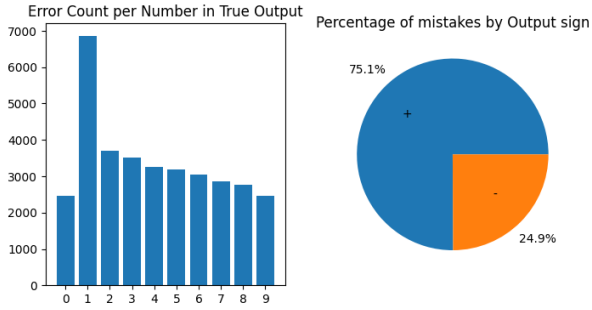


Figure 17: Mistake counts and ratios depending on output for the 10/90 data split model

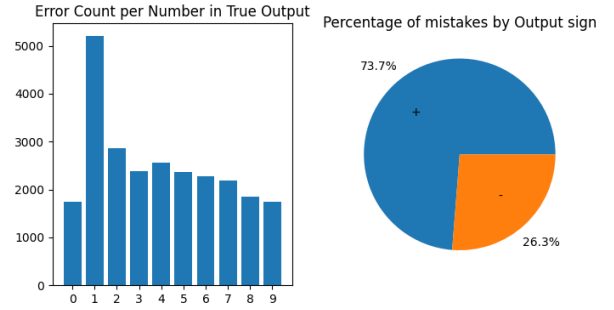


Figure 18: Mistakes counts and ratios depending on output for the deeper encoder model

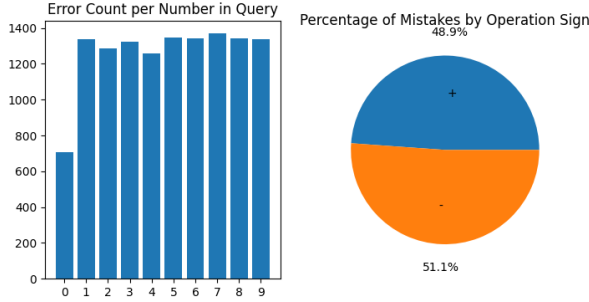


Figure 19: Mistake counts and ratios depending on input for the convolutional model

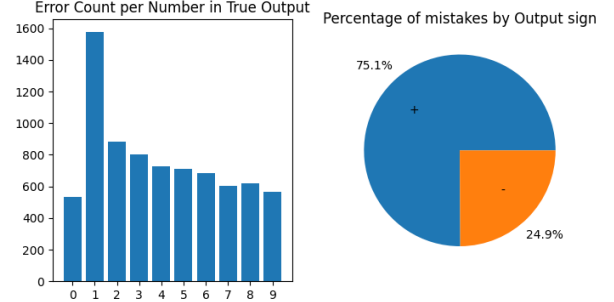


Figure 20: Mistakes counts and ratios depending on output for the convolutional model

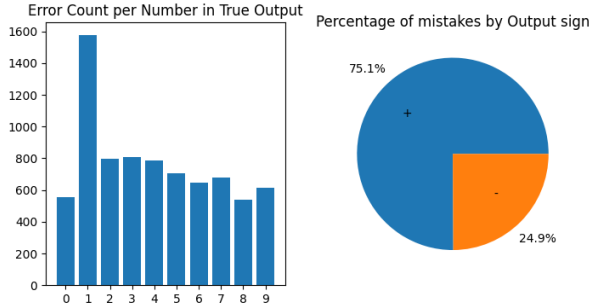


Figure 21: Mistake counts and ratios depending on input for the flattened images model

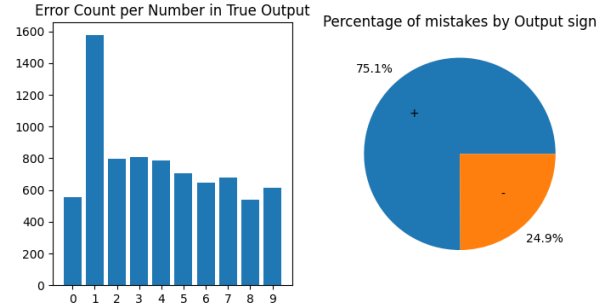


Figure 22: Mistakes counts and ratios depending on output for the flattened images model

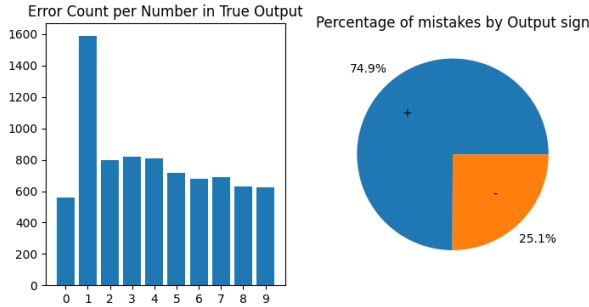


Figure 23: Mistake counts and ratios depending on input for the deeper encoder model

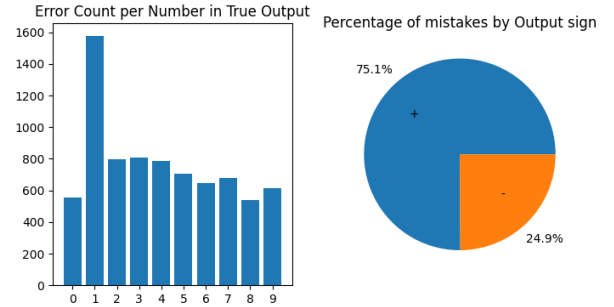


Figure 24: Mistakes counts and ratios depending on output for the deeper encoder model

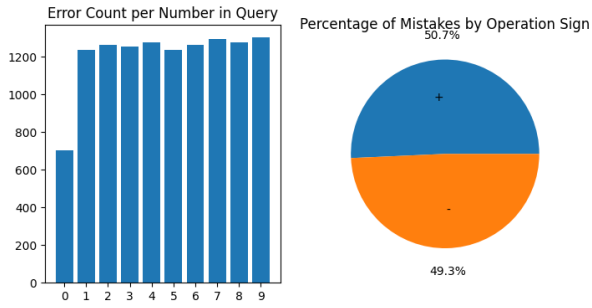


Figure 25: Mistake counts and ratios depending on output for the text-to-image model

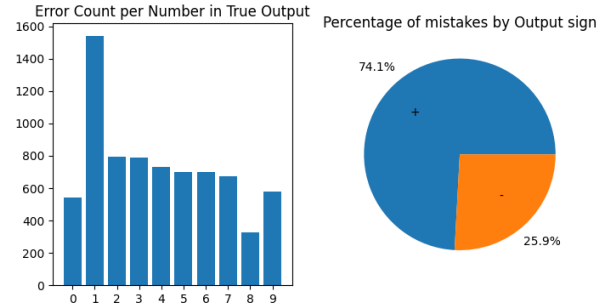


Figure 26: Mistake counts and ratios depending on output for the text-to-image model

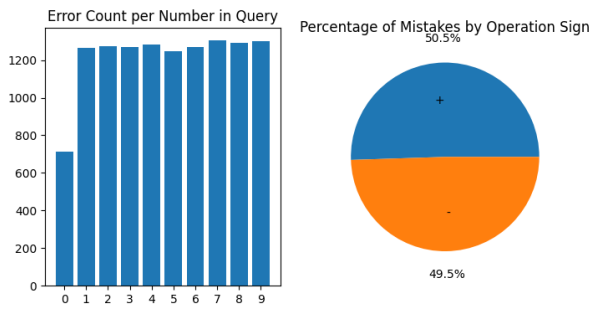


Figure 27: Mistake counts and ratios depending on output for the deeper encoder model

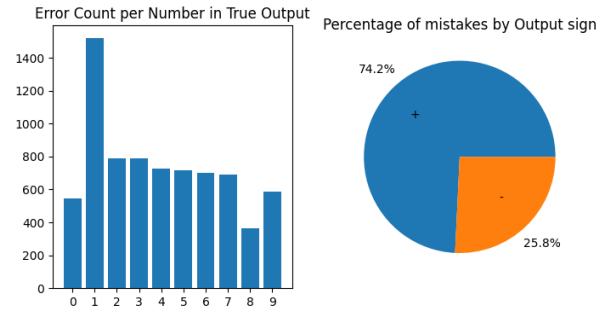


Figure 28: Mistakes counts and ratios depending on output for the deeper encoder model

4 Contributions

Yutao Liu: Task 1 code and report
 Nael Belhaj: Task 2 code and report